Renewable- and Thermal-Aware Hierarchical MARL Workflow Scheduler

Project summary and GitHub repository guide | Mohammad Sepehr Ansari | Supervisor: Dr. Mohammad Reza Rasouli

1. Goal

Cloud data centers consume large amounts of electricity, and solar power is intermittent and differs between sites. The project builds a scheduler that decides, for every task of a workflow (a DAG of dependent tasks), **which data center and which server** should run it, so that brown (grid) energy and execution time are minimized while rack temperatures stay safe. It reproduces and extends the hierarchical multi-agent actor-critic framework of Jayanetti et al. (IEEE TPDS, 2024).

2. What was done and how

- **Simulator:** 10 data centers x 25 heterogeneous servers, with the paper's power, execution-time and energy models, driven by an event-driven clock (a task starts when all its predecessors finish).
- **Data realism:** hourly solar output from PVGIS for 10 real cities (instead of one trace for all sites); 1000 workflows with task sizes resampled from the Alibaba GPU-v2020 cluster trace and synthetic layered DAGs with Poisson arrivals.
- **Learning method:** a global agent picks the data center and local agents pick the server (two 128-unit hidden layers, Adam, 150 episodes x 20 workflows). Critics see only a local neighborhood of information, which keeps training scalable. Compared against Random, Green-Opt (greedy green-energy-first) and Common-Actor (one shared critic).
- **Thermal extension:** a lumped-capacitance rack temperature model, a hotspot penalty in the reward and, optionally, rack temperatures in the agents' state, tested in a controlled 3-condition ablation (no thermal awareness / penalty only / penalty + observability).

3. Results

Method	Energy / task (J)	Exec. time / task (s)	Mean global reward	Wait time (s)
Random	2809.5	4.71	-0.941	0.036
Green-Opt	1819.7	3.72	-0.548	0.000
Common-Actor	699.8	1.45	-0.418	0.336
Proposed (hierarchical MARL)	**364.8**	**0.73**	**-0.395**	**0.135**

The proposed scheduler uses 87.0%, 79.9% and 47.9% less energy per task than Random, Green-Opt and Common-Actor, with the shortest execution time. In the thermal ablation, the share of episodes with a severe hotspot (> 40 C) fell from 100% to **0%** only when the penalty was combined with rack-temperature observability (mean peak 33.6 C vs. 44.3 C; Welch t = -16.1, p = 2e-13); the penalty alone barely helped (85%). Not reproduced: the paper's faster-convergence claim for the local-neighborhood critic. Results are simulation-based.

4. How the GitHub repository works

src/ holds the code by phase (environment, solar data, workflow generator, agents, training loop, baselines, thermal model); **thesis_pipeline_phase0_to_4_5.ipynb** runs the whole experiment step by step with outputs; **scripts/** regenerates the workflows and the article's tables; **results/** stores histories, summary tables and trained checkpoints; **data/** receives downloaded inputs; **tests/** has fast smoke tests; **docs/ARCHITECTURE.md** explains the design.

Quick start: pip install -r requirements.txt -> python scripts/summarize_results.py (reproduces the tables in seconds) -> pytest -q -> python scripts/generate_workflows.py, then run the notebook from the repo root to retrain (about 7 minutes per 150-episode run, CPU only).